

misc.lg/2_cent_tip_piping_to_gnu_plot_from_c.html (or <http://goo.gl/xcej>).

There is also an ANSI C interface to Gnuplot available at <http://ndevilla.free.fr/gnuplot/>, but I haven't personally used it.

GNU Octave

GNU Octave (referred to as Octave from now on) is a software tool intended for numerical computation. Octave has extensive tools to solve common numerical linear algebra problems, to find the roots of non-linear equations, integrate ordinary functions, manipulate polynomials, and integrate ordinary differential and differential-algebraic equations. Octave is available for use on Linux, Solaris, Mac OSX, and Windows. If your study or play involves numerical computations, Octave is for you. Octave has been designed with MATLAB compatibility in mind, so with some careful design you could write scripts that run on both MATLAB and Octave. It is easily extensible and customisable via user-defined functions written in Octave's own language, or by using dynamically loaded modules written in C++, C, FORTRAN, or other languages.

Octave is most definitely available in the package repository of your Linux distribution, and that is the easiest way to get it.

GNU Octave comes with extensive documentation, which is the best place to get started. See <http://www.gnu.org/software/octave/doc/interpreter/> or <http://goo.gl/ew20>.

If you're interested in Octave (and happen to like my writing!), I've written a series of articles on Octave for this magazine. You can obtain PDFs of the articles from my blog post at <http://amitkaha.wordpress.com/2009/11/30/article-series-getting-started-with-gnu-octave/> (or <http://goo.gl/Dkpm>)

Add Python to your toolkit

I will not use this section of the article to advocate the use of Python. Rather, I would suggest you consider using Python in your research or think of using it in the future as just another one of your many experiments. Python is already installed on most Linux distributions, and you can use Python in many ways, including as a command-line scientific calculator, based on the *math* module (whose documentation is at <http://docs.python.org/library/math.html>). For example:

```
$ python
Python 2.6.4 r(264:75706, Dec 7 2009, 18:45:15)
[GCC 4.4.1] on linux2
Type "help", "copyright", "credits" or "license" for more information.
>>> import math
>>> math.factorial(10)
3628800
>>>
```

As you can guess, you need to import the *math* module every time you want to use it. You can avoid doing this by automatically importing it at start-up.

1. Create a file: *.pythonrc* in your *\$HOME* (user's home

folder) and add the following line:

```
import math
```

2. Now, in your *.bashrc* or similar file (that is executed when you log in to your account), add:

```
export PYTHONSTARTUP=$HOME/.pythonrc
```

After your next login, every time you start Python interactively, you should have the *math* module already imported and ready to use:

```
$ python
Python 2.6.4rc1 (r264rc1:75270, Oct 10 2009, 02:40:56)
[GCC 4.4.1] on linux2
Type "help", "copyright", "credits" or "license" for more information.
>>> math.pi
3.1415926535897931
```

The decimal module (documentation at <http://docs.python.org/library/decimal.html>), another interesting module, provides support for decimal fixed-point and floating-point arithmetic. You can use the fractions module (see <http://docs.python.org/library/fractions.html>) for rational number arithmetic.

Here, the beauty is that you do not need to know the Python programming language constructs to use these modules; just fire up the Python interpreter, import the module(s) you want to use, and call the appropriate function!

Scientific libraries

If you are already conversant with Python, then you would definitely like to know about scientific libraries that you may find useful in your research:

- **NumPy and SciPy:** NumPy (<http://numpy.scipy.org/>) is an extension to the standard Python programming language, adding support for large multi-dimensional arrays and matrices. SciPy (<http://www.scipy.org/>) builds upon NumPy to provide various mathematical functions for integration, optimisation, interpolation, linear algebra, statistics and others.
- **matplotlib:** (see <http://matplotlib.sourceforge.net/>) This is a plotting library for Python, which is also used by a lot of other Python scientific tools for plotting data.
- **NetworkX:** NetworkX (<http://networkx.lanl.gov/>) is a Python library for the creation, manipulation, and study of complex networks. Even if you do not have the need to build very complex networks, this is fun to play around with to create simple graphs too.

Beam it with Beamer!

You have done your experiments, sent in your findings, they've been accepted, and now you are going to present them at a conference. You want to prepare some slides to keep